INT202 Complexity of Algorithms

Tutorial 2

Q1. Compute Big-Oh notation.

a)
```
void func(int n){
 int i = 0;
 while(i * i * i <= n)
   i++; }
```

$i^3 \le n$

$\Rightarrow i \le \sqrt[3]{n}$

$O(n^{\frac{1}{3}})$

b)
```
void recursive(int n, int m, int o) {
 if (n <= 0) {
   printf("%d, %d\n", m, o);
 } else {
   recursive(n - 1, m + 1, o);
   recursive(n - 1, m, o + 1); }}
```

$n=3$

$f(n) = 2f(n-1)$

$= 2^2 f(n-2)$

$= 2^n f(n-n)$

$= 2^n f(0)$

$\because f(0)$ is constant

$\therefore O(2^n)$

c)
```
void func(int n) {
 for (int i = 1, s = 0; i <= n; ++i) {
      int t = 1;
      for (int j = 1; j <= i; ++j)
            t = t * j; }}
```

$O(n)$

$i=1 \quad j=1$

$2 \quad 1, 2.$

$3 \quad 1, 2, 3.$

$O(1+2+3+\ldots+n)$

$= O\left(\frac{n(1+n)}{2}\right)$

$O(n) \times O\left(\frac{n^2+n}{2}\right) = O\left(\frac{n^2}{2}\right) + O(n)$

$= O(n^2)$

Q2. State whether the following statement is true or false (given n is the size of input), and give a brief justification for your answer.

a) If $f(n)$ and $g(n)$ are both $O(h(n))$, then $f(n)+g(n)$ is $O(h(n))$. T

b) If $f(n) = \sum_{i=1}^{n} \log i$, then $f(n)$ is $O(\log n)$ F

Q3. Consider the following code fragment.

```
for i=1 to n do
     for j=i to 2*i do
          output hello
```

$1, 2, 3 \ldots n$

$1, 2 \quad 2, 3, 4 \quad 3, 4, 5, 6 \quad = i$

Let T(n) denote the number of times 'hello' is printed as a function of n.

1) Express T(n) as a summation;

2) Simplify the summation and give the worst-case running time using Big-Oh notation.

Q2 (a) True,

If $f(n) = O(h(n))$, then exist $n_1, c_1 > 0$

that $f(n) \leq c_1 \cdot h(n)$ for all $n \geq n_1$

similarly, if $g(n) = O(h(n))$, then exist $n_2, c_2 > 0$

that $g(n) \leq c_2 \cdot h(n)$ for all $n \geq n_2$.

$\therefore$ for all $n_3 \geq n_0$,

$$f(n) + g(n) \leq (c_1 + c_2) h(n)$$

$$\Rightarrow f(n) + g(n) = O(h(n))$$

(b) False

$$f(n) = \lg 1 + \lg 2 + \ldots + \lg n = \lg(1 \times 2 \times \ldots \times n) = \lg(n!)$$

$$\leq \lg(n^n) = n \lg n$$

$$\therefore O(n \lg n)$$

Q3 (2) $\sum_{i=1}^{n} (i+1) = \frac{n(2+n+1)}{2} = \frac{n^2+3n}{2}$

$$\Rightarrow O(n^2).$$

(1) $\sum_{i=1}^{n} \sum_{j=i}^{2i} (1) = \sum_{i=1}^{n} (2i - i + 1)$

24 Tut 2.